

BiblioTech

SOEN 363

Mohammed Al Assaad
Ziad Elsharkawi
Adam Chami
Vinisha Manek
(T31)



Project Topic - Books

Motivation

- Structured & Universally Relatable
- Versatile topic

User Benefits

- Price Information
- Ebooks Links

Datasources

- Google Books API
 - Commercial book details
 - Rate limiting
- Open Library API
 - No API key required
 - Metadata



Outline

Relational

ER Data Model

Database Metrics

Domains, Types

NoSQL

NoSQL Data Model

Design Changes

Domains, Types

Migration

Direct Transfer

Indexes

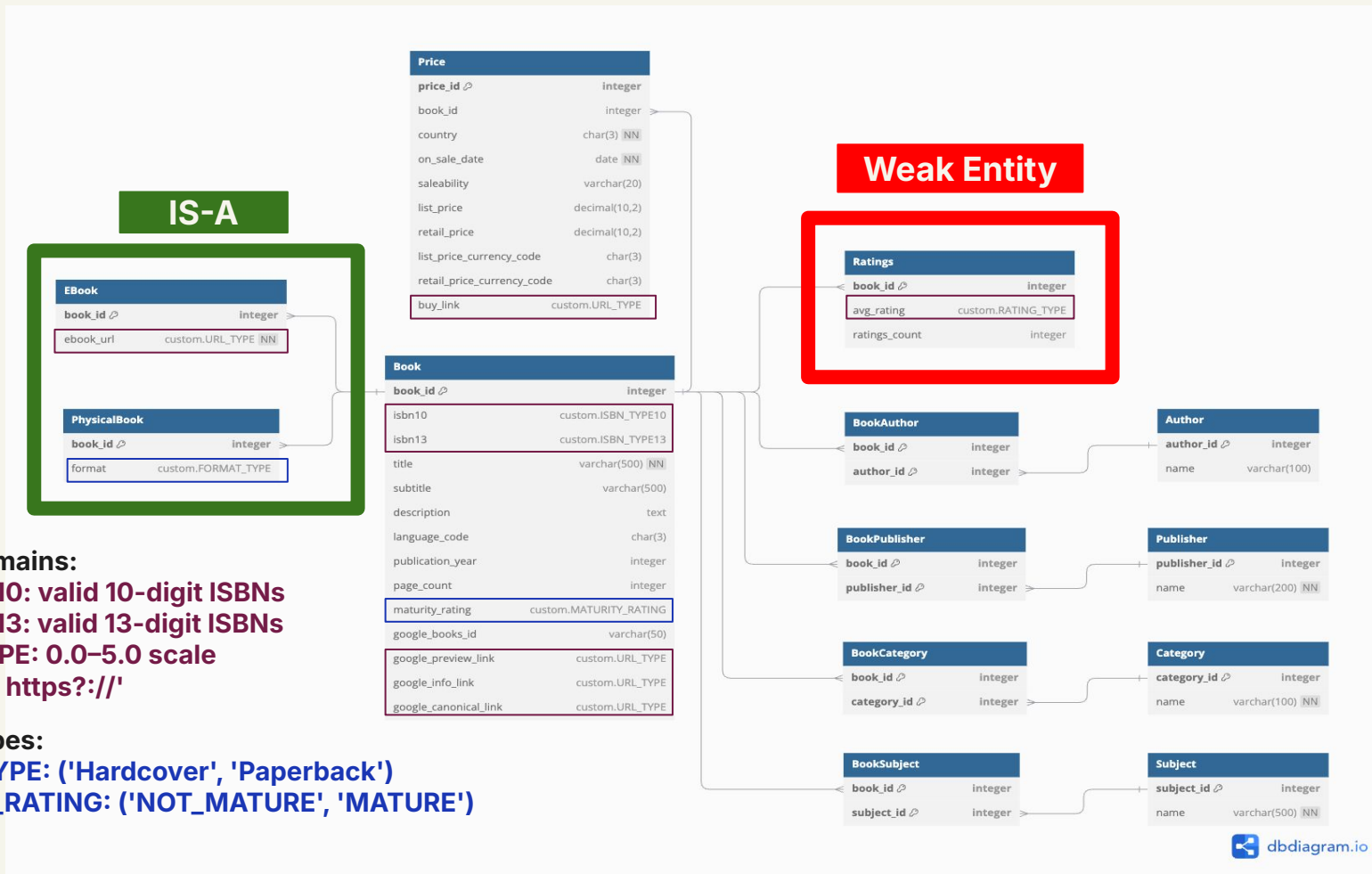
Query
Demonstration

Challenges & Solutions

Future Advice

Phase 1 - Relational Database

- PostgreSQL
- Database Design
 - IS-A relationships (e.g., PhysicalBook, EBook)
 - Weak Entity (Ratings)
 - Complex referential integrity via triggers
 - Domains and types for ISBNs, ratings, URLs
- Database Size: 487 MB

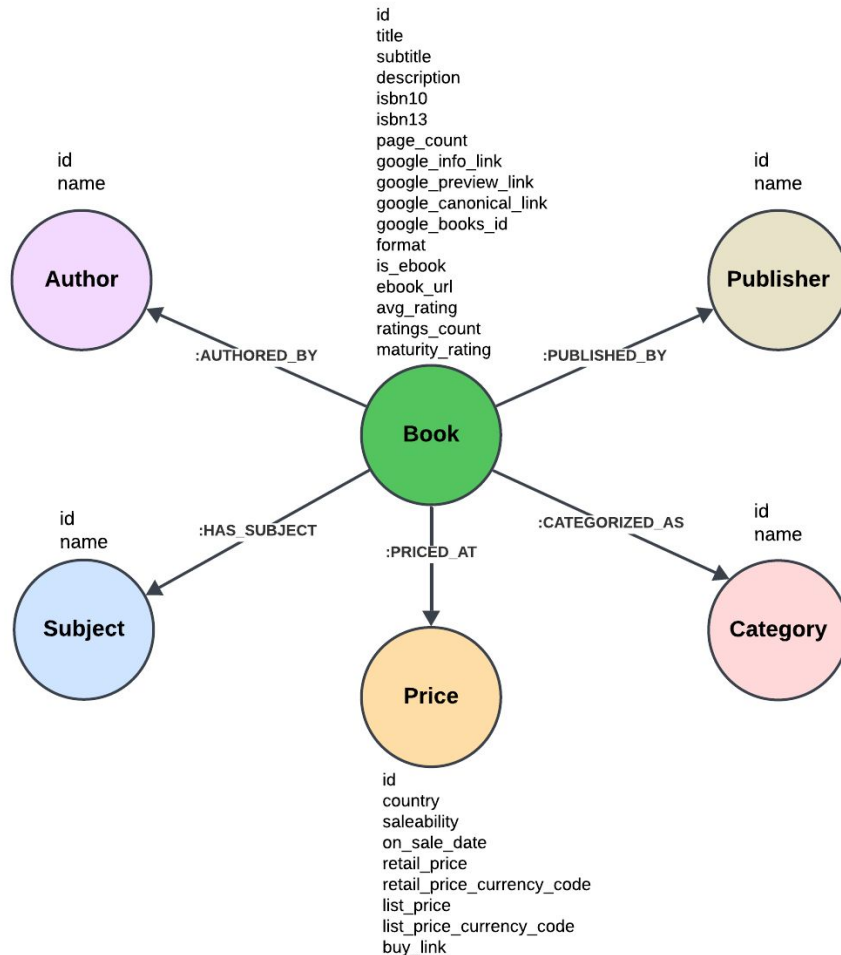
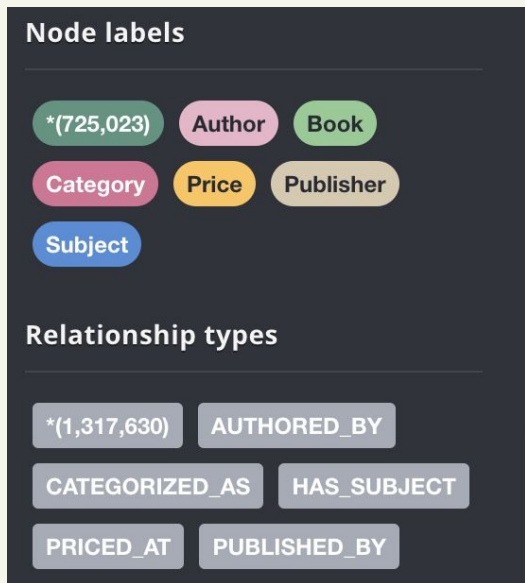


Phase 2 - Transition to NoSQL

- Database Selection: Neo4j
- Design Adjustments
 - Removal of both IS-A relationships
 - Added Ratings (weak entity) into Book

Store Size	
Total	4.48 GiB
Database	2.23 GiB

Neo4j Graph Data Model



Migration Process (Relational → NoSQL)

- Direct transfer (no intermediate used)
- Index creation

```
# postgresQL connection
pg_conn = psycopg2.connect(
    host=os.getenv("DB_HOST"),
    database=os.getenv("DB_NAME"),
    user=os.getenv("DB_USER"),
    password=os.getenv("DB_PASSWORD"),
    port=os.getenv("DB_PORT")
)
pg_cursor = pg_conn.cursor()

# neo4j connection
NEO4J_URI = os.getenv("NEO4J_URI")
NEO4J_USERNAME = os.getenv("NEO4J_USERNAME")
NEO4J_PASSWORD = os.getenv("NEO4J_PASSWORD")
driver = GraphDatabase.driver(NEO4J_URI, auth=(
```

```
def create_indexes(session):
    """Create indexes for all node types to optimize relationship creation"""
    print("Creating indexes...")
    session.run("CREATE INDEX book_id IF NOT EXISTS FOR (b:Book) ON (b.id)")
    session.run("CREATE INDEX price_id IF NOT EXISTS FOR (p:Price) ON (p.id)")
    session.run("CREATE INDEX author_id IF NOT EXISTS FOR (a:Author) ON (a.id)")
    session.run("CREATE INDEX publisher_id IF NOT EXISTS FOR (p:Publisher) ON (p.id)")
    session.run("CREATE INDEX category_id IF NOT EXISTS FOR (c:Category) ON (c.id)")
    session.run("CREATE INDEX subject_id IF NOT EXISTS FOR (s:Subject) ON (s.id)")
```


Transferring Relationships (Relational → NoSQL)

- Dynamic relationship creation
- Batch processing using UNWIND

```
def create_relationships(tx, relationships, rel_type, node_type):  
    tx.run(f"""  
        UNWIND $rels AS rel  
        MATCH (b:Book), (e:{node_type})  
        WHERE b.id = rel.book_id AND e.id = rel.entity_id  
        CREATE (b)-[:{rel_type}]->(e)  
        """, rels=relationships)
```

Query Demonstration!

Query 1:

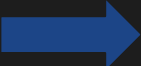
Counting the number of Hardcover, Paperbacks and Ebooks
(Aggregate function)

Query 2:

Finding books published in 2020, and onwards, with either
"Databases" or "Computer Science" as subjects

Query Demonstration (Indexes)

```
vinishamaneke@vinishas-MacBook-Pro SOEN363 % python3 execute_cypher.py  
dropping existing indexes...
```

 before creating indexes:

```
basic search on attribute value: execution time: 0.1117179394 seconds  
aggregation paperback: execution time: 0.0477931499 seconds  
aggregation hardcover: execution time: 0.0316610336 seconds  
aggregation ebook: execution time: 0.0489740372 seconds  
top n entities satisfying a criteria, sorted by an attribute: execution time: 0.0732951164  
group books by year of publication: execution time: 0.0575559139 seconds
```

creating indexes...

```
created full-text index: book_title_index  
created index: book_year_index  
created index: book_format_index  
created index: book_lang_pages_index  
created index: book_ebook_index
```

 after creating indexes:

```
basic search on attribute value: execution time: 0.0622220039 seconds  
aggregation paperback: execution time: 0.0423910618 seconds  
aggregation hardcover: execution time: 0.0309059620 seconds  
aggregation ebook: execution time: 0.0347411633 seconds  
top n entities satisfying a criteria, sorted by an attribute: execution time: 0.0493969917  
group books by year of publication: execution time: 0.0388028622 seconds
```

Challenges

&

Future Suggestions

- Google Books API: rate limiting
- Separate ISBNs - Physical & Ebook
- Migration Space Issue

- Adding editions info
- Better ratings info (Goodreads API)
- Merging physical and ebook
- Adding audio books

```
# Limit the amount of memory that all of the running
# The default value is 70% of the heap size limit.
dbms.memory.transaction.total.max=2048m
```

```
*****
# Other Neo4j system properties
*****

dbms.memory.heap.initial_size=2G
dbms.memory.heap.max_size=4G
dbms.memory.pagecache.size=512m
```

Thank you for listening!